Technical Jargon Simplifier

COURSE PROJECT REPORT
Submitted by

Avinash
ASI23CA024

Under the guidance of

Dr.SARIKA S

To
APJ Abdul Kalam Technological University
In partial fulfillment of the requirements for the award of the Degree of
Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence)

Department of Artificial Intelligence and Data Science

ADI SHANKARA INSTITUTE OF ENGINEERING AND TECHNOLOGY, KALADY

OCTOBER 2026

Technical Jargon Simplifier

COURSE PROJECT REPORT
Submitted by

Avinash
ASI23CA024

Under the guidance of

Dr.Sarika S

To

APJ Abdul Kalam Technological University
In partial fulfillment of the requirements for the award of the Degree of
Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence)



Department of Artificial Intelligence and Data Science

ADI SHANKARA INSTITUTE OF ENGINEERING AND TECHNOLOGY, KALADY

OCTOBER 2026

ADI SHANKARA INSTITUTE OF ENGINEERING AND TECHNOLOGY, KALADY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)



CERTIFICATE

Certified that this is a bonafide record of the course project entitled

Technical Jargon Simplifier

Submitted by

Avinash
ASI23CA024

during the year 2026-27 in partial fulfillment of the requirement for the award of the degree of
Bachelor of Technology in Computer Science (Artificial Intelligence) and Engineering

Faculty in Charge

ACKNOWLEDGMENT

I express my sincere gratitude to Dr. Sarika S for her guidance and support during this course project. Her feedback helped refine the study of basic natural language processing and the presentation of a beginner-friendly technical-text simplifier.

I thank the Department of Artificial Intelligence and Data Science, Adi Shankara Institute of Engineering and Technology, Kalady, for the academic environment supporting this work. I also acknowledge the maintainers of Python and Jupyter for the open-source tools used in the project.

Avinash
ASI23CA024

ABSTRACT

Technical Jargon Simplifier is a beginner-friendly Python NLP project that explains recognized technical terms using a small custom glossary and visible sentence rules. It accepts text, normalizes it, tokenizes it, detects glossary terms and produces a simpler reading accompanied by full definitions. The glossary contains 42 entries across artificial intelligence, computer science, networking and programming.

The implementation uses only Python standard-library modules for the engine and optional local browser interface. A self-contained Jupyter Notebook contains the complete glossary, processing functions, 18 demonstrations, editable user input and evaluation. It requires no trained model, large dataset, paid API, cloud service or GPU. Stopwords are removed only in an inspection view so that sentence structure and negation are retained during rewriting.

All 12 code cells executed successfully from an isolated working directory. Evaluation passed 18 handcrafted cases by checking exact expected-term detection and definition inclusion in the complete explanation. Sixteen engine unit tests, five HTTP boundary checks and 12 browser checks also passed. This case success rate measures specific rule behavior; it does not measure human comprehension, semantic preservation or general accuracy. The report describes the methodology, notebook organization, implementation, sample outputs, UI screenshots and limitations.

TABLE OF CONTENTS

Sl. No.	Contents	Page No.
1	**Abstract**	5
2	**Chapter 1: Introduction**	7
2.1	Problem Statement	7
2.2	Objectives	7
2.3	Scope of the Project	7
3	**Chapter 2: Methodology / System Design**	8
3.1	Proposed Methodology	8
3.2	Tools and Technologies Used	9
3.3	System Architecture / Workflow	9
4	**Chapter 3: Implementation and Results**	11
4.1	Implementation Details	11
4.2	Results and Discussion	11
4.3	Testing and Screenshots	13
5	**Chapter 4: Conclusion and Future Scope**	18
	References	19

CHAPTER 1 INTRODUCTION

Technical writing often assumes familiarity with specialist vocabulary. A beginner may understand the overall topic but struggle with terms such as tokenization, latency or authentication. A small glossary-based system can make those terms easier to approach while making its own behavior transparent and inspectable.

1.1 Problem Statement

Build a Python program that accepts a technical sentence or short paragraph, identifies supported jargon and produces an accessible explanation using a custom glossary and basic NLP rules. It should display the original text, recognized terms, definitions and simplified output, while handling empty text and input without recognized terms.

1.2 Objectives

The objectives are to maintain at least 30 glossary entries, implement normalization and tokenization, apply stopword filtering only where appropriate, recognize longer technical phrases before shorter matches, generate rule-based rewrites and demonstrate the complete project in Jupyter Notebook. A minimal local UI should make the same engine available without adding a complex application stack.

1.3 Scope of the Project

The implemented glossary contains 42 entries across four subject areas. Aliases cover supported plurals, abbreviations and spelling variants. The engine uses a single substitution pass, a short difficult-word list, a limited learning-pattern sentence rewrite and basic article cleanup. It is a deterministic rule system, not a trained statistical or advanced AI model.

The notebook is the primary academic deliverable and runs independently of the other project files. The optional UI uses Python's built-in HTTP server and one HTML file. There is no database, authentication, external service, React or Streamlit. Text remains in the local running process and browser; the application does not persist pasted content.

CHAPTER 2 METHODOLOGY AND SYSTEM DESIGN

2.1 Proposed Methodology

The workflow validates input type, normalizes text, creates inspection tokens, detects technical terms with a combined regular expression and processes each sentence using explicit substitution rules. The returned result contains the original input, normalized text, simplified reading, term counts, full definitions and preprocessing details.

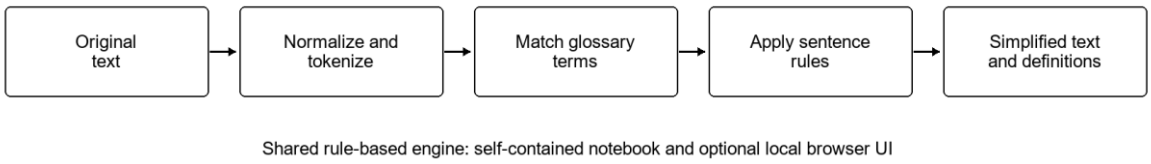


Figure 2.1 Rule-based jargon detection and simplification workflow

2.2 Custom Technical Glossary

Each glossary entry contains a canonical term, category, recognized aliases, a short substitution phrase and a full definition. Noun entries with plural forms also store a plural substitution phrase. This separates a compact sentence replacement from the more complete beginner explanation shown below it.

Subject area	Entries	Example terms
AI	12	machine learning, neural network, tokenization
Computer science	10	algorithm, database, encryption
Networking	10	bandwidth, latency, router
Programming	10	API, compiler, function

The glossary is authored for the project and is not a downloaded training dataset. Every glossary entry occurs in at least one technical demonstration. Terms with context-dependent meanings, such as function or exception, are interpreted according to the technical glossary rather than through contextual language understanding.

2.3 Tools and Technologies Used

Tool	Purpose
Python standard library	Core engine and local server
re and unicodedata	Regex matching, tokenization and normalization
http.server and json	Local browser requests and structured responses
Jupyter Notebook	Self-contained academic implementation and demonstrations
HTML, CSS and JavaScript	Minimal responsive interface
unittest	Engine regression tests

requirements.txt contains Jupyter Notebook for local notebook use. The engine and local UI require no third-party Python packages. Colab already supplies a notebook environment, so it does not need a package-install cell for this project. Development verification used existing notebook execution and browser-testing tools; these are not runtime dependencies.

2.4 Preprocessing and Technical Term Detection

Normalization applies Unicode NFKC, standardizes selected quotation marks and dashes and collapses repeated whitespace. A regex tokenizer produces lowercase words, numbers and punctuation for inspection. A small stopword set removes low-information words only from content_tokens; the original sentence words and negation remain available for rewriting.

The matcher tries longer glossary aliases first and uses word boundaries to prevent matches inside unrelated words. Spaced and hyphenated forms are handled consistently. Repeated occurrences are counted under one canonical entry. The same combined pattern performs a single replacement pass so newly inserted phrases are not detected and expanded again.

2.5 System Architecture and Workflow

simplifier.py contains the glossary, preprocessing, matching, sentence rules and manually labeled examples. app.py exposes a bounded local JSON endpoint and serves index.html. The browser displays a side-by-side reading workspace, full definitions, preprocessing inspection and an expandable glossary. The notebook embeds the same functions and data so that it has no project-file imports.

2.6 Sentence Rules

The engine splits text at sentence-ending punctuation, substitutes short glossary phrases and rewrites a small difficult-word list, such as utilize to use. A specific learning-pattern rule turns a subject using training datasets to optimize predictive models into a learn-from-examples explanation. Singular/plural phrase selection and article cleanup improve basic grammar, but there is no general grammatical or semantic model.

NOTEBOOK ORGANIZATION

The notebook contains 22 cells, of which 12 are executable Python cells. It was executed from beginning to end using a clean working directory that did not contain the project modules. All code cells completed without errors, demonstrating that the notebook is self-contained.

Required section	Included content
Project introduction	Objective, author and rule-based scope
Imports and setup	Only re and unicodedata for the engine
Technical glossary	42 entries and subject-area count
Text preprocessing	Normalization, tokens and content tokens
Technical term detection	Aliases, word boundaries and longest-phrase matching
Text simplification	Sentence rules and structured result
Sample demonstrations	16 technical examples, plain text and empty input
User input and testing	Editable USER_TEXT; optional interactive input disabled
Evaluation and conclusion	Expected-term checks, definition inclusion and limitations

The default notebook path never waits for input(). A student edits USER_TEXT and reruns the user-input cell. Interactive console entry is available through an explicit opt-in flag. The notebook functions were structurally compared with the local engine and match, reducing the risk that the UI and notebook demonstrate different logic.

To run locally, install requirements.txt, start python -m notebook and open Technical_Jargon_Simplifier.ipynb. In Google Colab, upload the notebook and choose Runtime, Run all. To use the optional UI, run python app.py and open http://127.0.0.1:8527 in a browser.

CHAPTER 3 IMPLEMENTATION AND RESULTS

3.1 Implementation Details

The project is maintained as an independent local Git repository for Avinash, ASI23CA024. The intended repository URL is https://github.com/AVINASH-V-BHASKARAN/Technical_Jargon_Simplifier. The URL was supplied for the report; publication is pending and no remote push was performed. Local notebook, API and UI verification took place on 15 October 2026.

File	Responsibility
Technical_Jargon_Simplifier.ipynb	Complete notebook, demonstrations and evaluation
simplifier.py	Shared glossary and deterministic NLP engine
app.py	Loopback-only local server and JSON input validation
index.html	Original text, simplified reading, definitions and NLP steps
test_simplifier.py	Unit and regression tests
requirements.txt and README.md	Notebook dependency and setup instructions

The local endpoint accepts a text field with at most 12,000 characters and a request body no larger than 64,000 bytes. Wrong input types and malformed requests receive useful errors. The UI renders text through textContent, provides keyboard submission and rejects stale responses after the user clears or edits the input.

3.2 Results and Discussion

Measurement	Observed result
Glossary size	42 entries across four subject areas
Notebook execution	12 of 12 code cells completed
Functional evaluation	18 of 18 cases passed (100.0%)
Glossary demonstration coverage	42 of 42 entries appear in the technical examples
Engine unit tests	16 passed
HTTP boundary checks	5 passed
Browser checks	12 passed

A case passes when the detected canonical-term set exactly matches its manually specified expected set and every detected term's full definition appears in the complete explanation. Ordinary-text and empty-input cases expect no detected terms. These handcrafted examples were used during development; they are not an independent held-out benchmark.

3.2.1 Example Simplification

Input: Machine learning algorithms utilize training datasets to optimize predictive models.

Simplified output: Machine learning methods learn from examples to make better predictions.

The detected entries are machine learning, algorithm, training dataset and predictive model. The sentence is rewritten by the explicit learning-pattern rule. Full definitions are separately included in the complete explanation and in the interface's detected-term cards.

3.2.2 Representative Demonstrations

Input	Detected terms
Machine learning algorithms utilize training datasets to optimize predictive models.	algorithm, machine learning, predictive model, training dataset
A neural network supports deep learning.	deep learning, neural network
Classification and regression use a dataset.	classification, dataset, regression
Overfitting reduces the value of inference.	inference, overfitting
Tokenization is used in natural language processing.	natural language processing, tokenization
The compiler checks source code.	compiler, source code
Debugging finds an exception in a function.	debugging, exception, function
Recursion and iteration can update variables.	iteration, recursion, variable
An API reads a database.	API, database
Binary search uses a data structure.	binary search, data structure

The notebook prints the actual simplified reading and each full definition for all 18 examples, including networking examples, ordinary text and empty input. The examples above illustrate detection coverage. They do not establish that every generated sentence is more readable or preserves every nuance.

3.2.3 Additional Behavior

A neural network is matched as a phrase rather than as unrelated substrings. API does not match inside snails. Repeated algorithms are counted under one canonical entry. Negation such as not remains in rewritten text and content tokens. For an ordinary sentence without recognized terms, the normalized wording is returned unchanged with an explicit explanation.

TEXT SIMPLIFICATION INTERFACE

3.3 Testing and Screenshots

The following image was captured from the running local interface. It shows the supplied machine-learning example, the generated simpler reading and the detected terms with their full definitions. The UI calls the result a glossary-based rewrite and invites review before use.

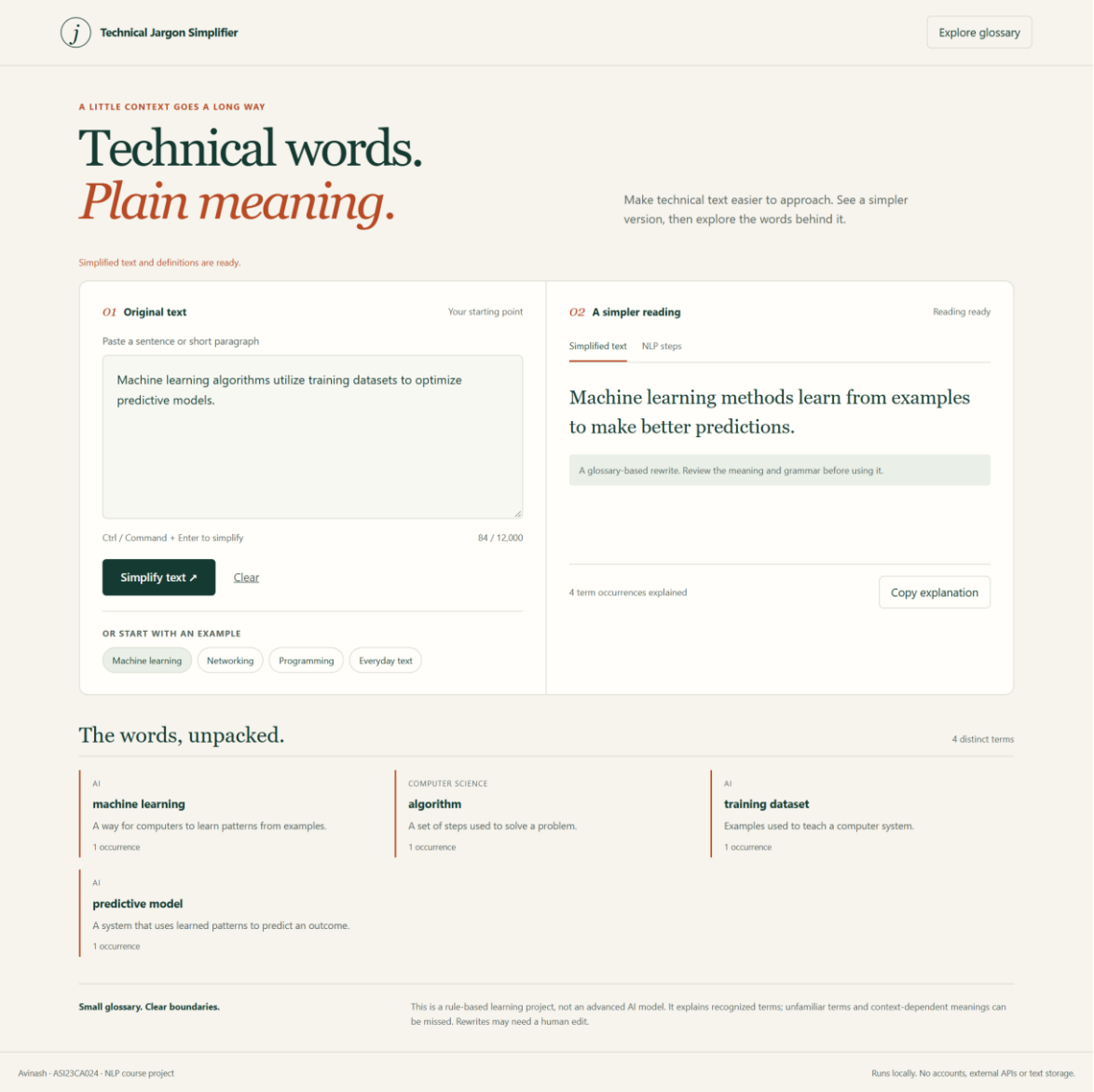


Figure 3.1 Fresh local screenshot of simplification and detected definitions

NLP PREPROCESSING INSPECTION

The NLP steps tab displays normalized text, tokens and the filtered content-token view. Stopword filtering is an inspection step; the full sentence remains the source for rewriting. The screenshot shows actual output from the Python engine.

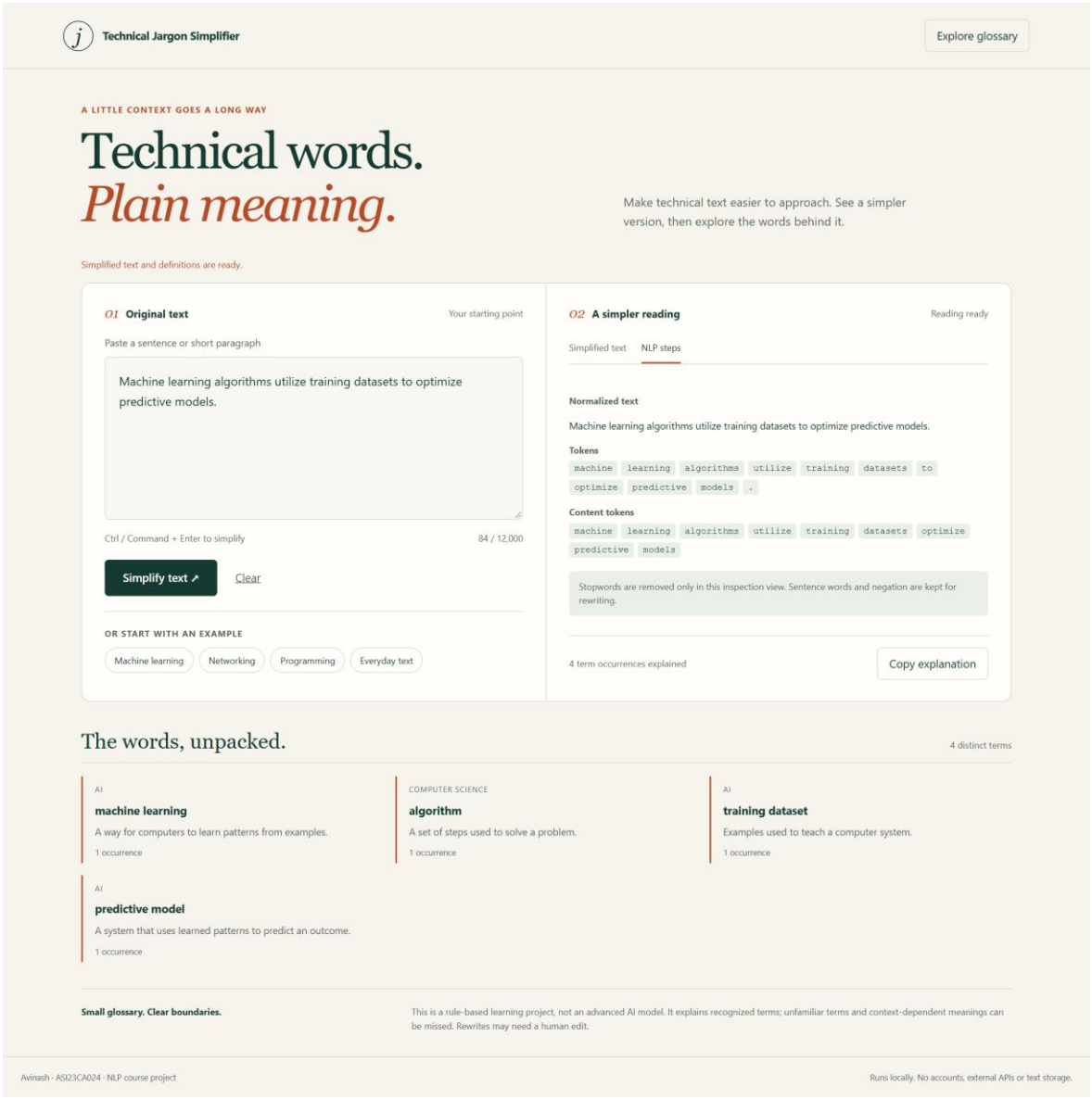


Figure 3.2 Fresh local screenshot of preprocessing inspection

NETWORKING EXAMPLE

The networking example recognizes router, packet and IP address, chooses the supported plural phrase for packets and shows the associated definitions. The output is a deterministic glossary expansion and may be longer than the input because an explanation can require additional words.

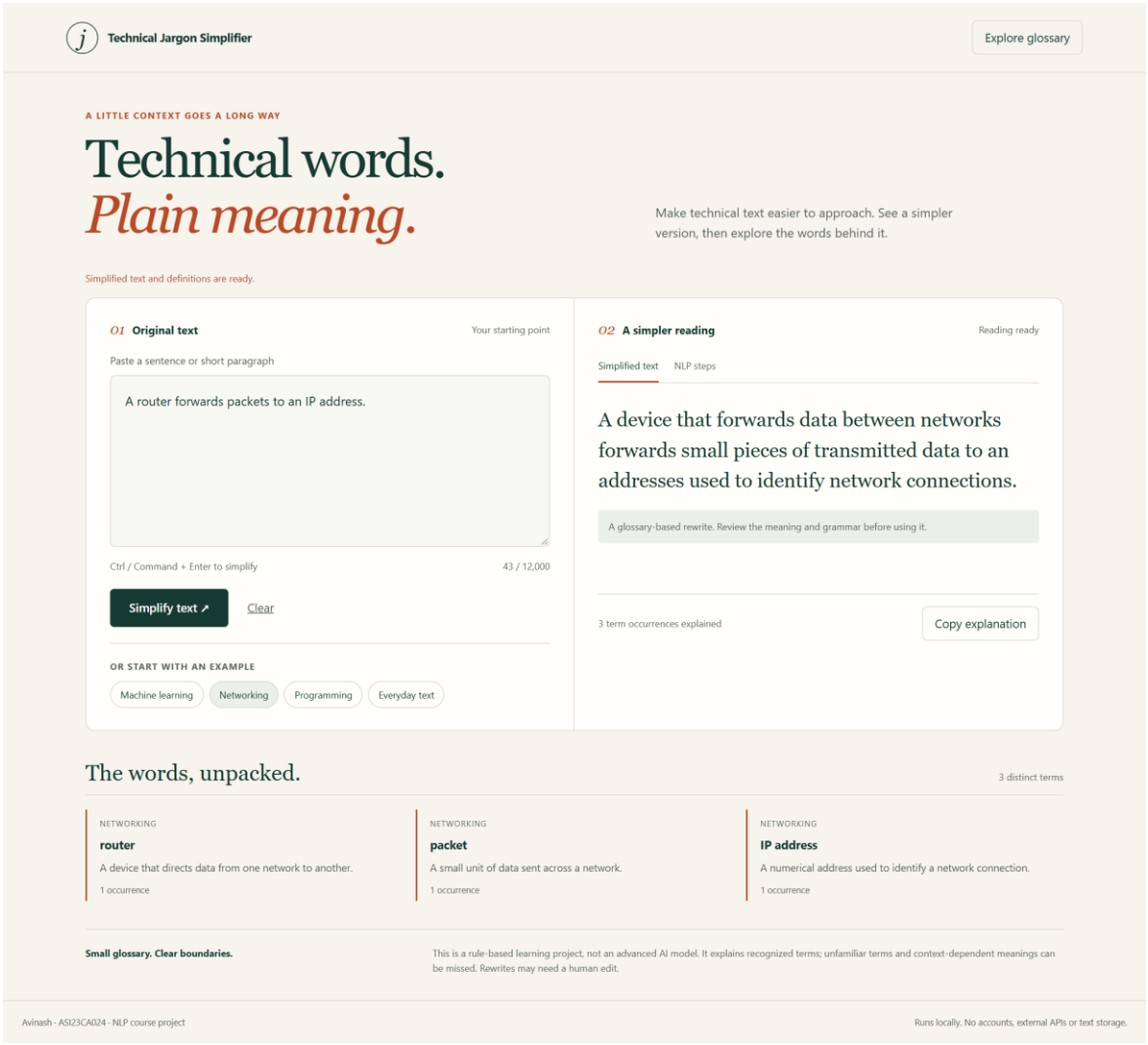


Figure 3.3 Fresh local screenshot of a networking example

INPUT WITHOUT RECOGNIZED TERMS

When the input contains no glossary terms, the application keeps the normalized wording and explicitly reports that no terms were recognized. It does not invent a technical interpretation. Empty input instead asks the user to enter text.

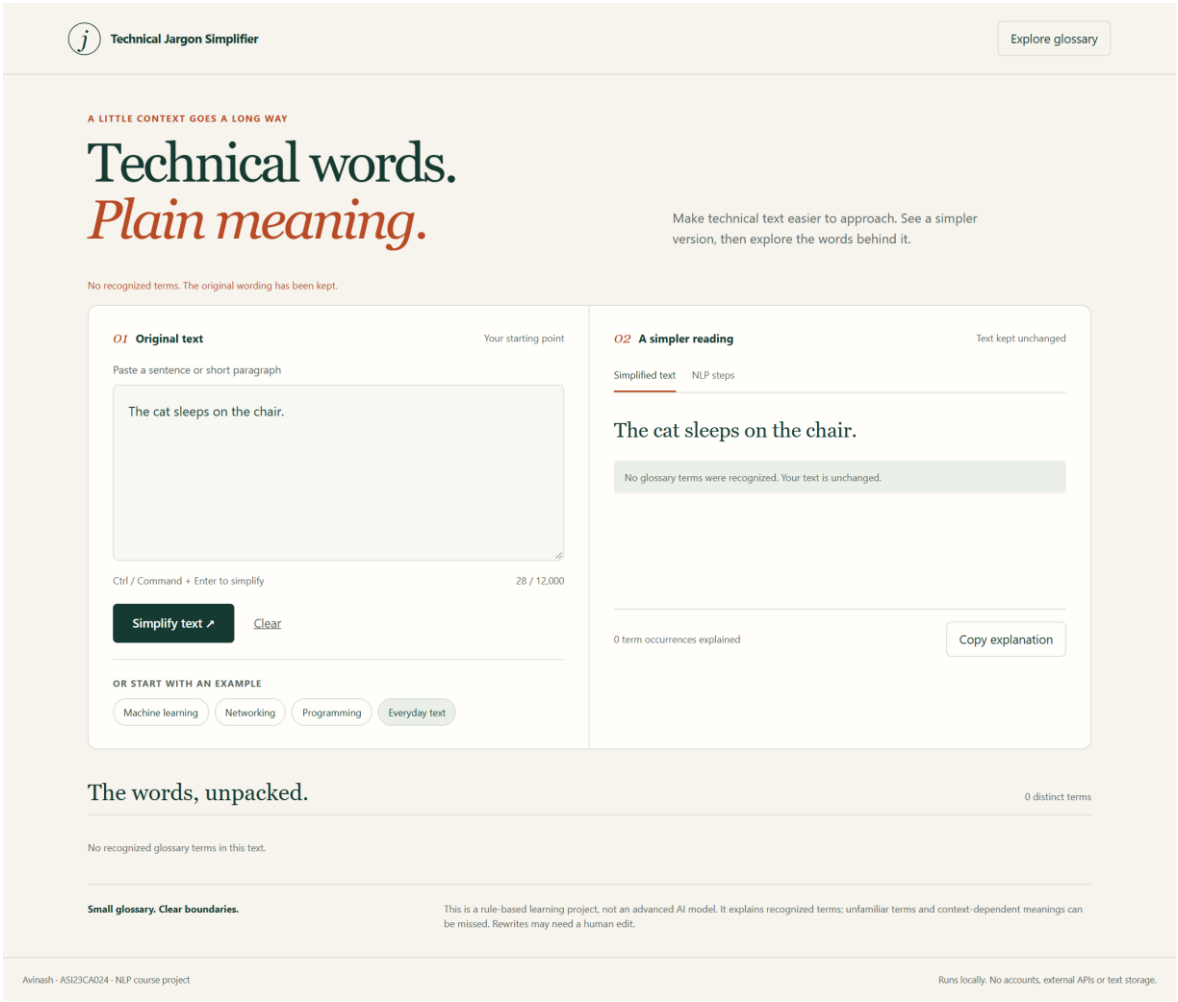


Figure 3.4 Fresh local screenshot of the no-recognized-term outcome

3.4 Software Verification

Area	Verification evidence
Engine	16 tests cover boundaries, phrase priority, negation, aliases, articles, singular/plural forms, Unicode and input handling
Notebook	All 12 code cells executed in an isolated directory; engine function bodies match the module
Case evaluation	18 expected-term and definition-inclusion cases passed
HTTP boundary	Valid input, empty input, wrong type, wrong shape and oversized text checked
Browser behavior	Real simplification, glossary, NLP steps, ordinary text, empty input, keyboard and safe rendering checked
UI resilience	Server error displayed and stale result discarded after clearing
Mobile layout	No horizontal overflow in tested 360 and 768 pixel views
Automated accessibility	Selected Axe WCAG A/AA rules passed on desktop and mobile result views

The 12 browser checks are named functional and accessibility checks, rather than a statistical sample of user behavior. Automated accessibility audits are limited to the tested views and rules. A read-only source review identified a singular/plural grammar issue; regression tests were added, the engine and notebook were synchronized and the revised checks passed.

3.5 Evaluation Limits

The reported 100.0% case success rate measures exact glossary detection and definition inclusion on examples designed for this vocabulary. Definition inclusion is a structural check and does not establish that a definition is pedagogically ideal. Human readability, factual preservation and unfamiliar vocabulary were not independently scored.

Some replacements can remain awkward, ambiguous or too long. A word such as function can have a non-programming meaning that the glossary cannot distinguish. Basic sentence boundaries can mishandle abbreviations. Article cleanup uses simple spelling rules rather than pronunciation or grammatical parsing. Output should be reviewed by a person.

CHAPTER 4 CONCLUSION AND FUTURE SCOPE

4.1 Conclusion

Technical Jargon Simplifier meets its educational objective through an inspectable glossary and basic NLP rules. It normalizes and tokenizes input, identifies supported technical phrases, produces a rule-based reading and displays complete definitions. The self-contained notebook runs in order, and the optional local UI reuses the same engine without introducing a complex application stack.

The project includes 42 glossary entries, 18 demonstrations and explicit handling of empty and ordinary-text inputs. Fresh checks verify the notebook, engine, HTTP boundary and exercised browser behavior. The observed success rate describes those functional cases and should not be presented as advanced AI capability or general simplification accuracy.

4.2 Limitations

The vocabulary is fixed and meanings are not inferred from context. Handwritten substitutions can change nuance or yield awkward grammar. Only explicitly listed aliases are recognized. The system has no learned model, broad paraphrasing ability, automatic fact checking or measured human-comprehension benefit. The intended GitHub publication remains pending.

4.3 Future Scope

Future improvements should begin with reviewed glossary additions and independent examples that include ambiguous meanings, abbreviations and complex sentence structures. Human readers can rate clarity and meaning preservation to complement structural tests. Rule changes should be accompanied by regression cases and kept synchronized between the engine and notebook.

More careful sentence segmentation and context-specific substitution rules could improve output while keeping the implementation understandable. The project should retain its small dependency footprint and explicit distinction between transparent rules and learned language models.

REFERENCES

[1] Avinash. Technical Jargon Simplifier. Intended repository: https://github.com/AVINASH-V-BHASKARAN/Technical_Jargon_Simplifier. Local implementation verified on 15 October 2026; publication pending.

[2] Project notebook: Technical_Jargon_Simplifier.ipynb. Self-contained glossary, implementation, sample demonstrations and evaluation.

[3] Project source and tests: simplifier.py, app.py, index.html and test_simplifier.py. Local verification on 15 October 2026.

[4] Python Software Foundation. Python standard-library modules re, unicodedata, json, http.server and unittest. Module documentation is available through the Python documentation and help().